Assignment 8: I/O Techniques

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

August 12, 2026

Deadline: **Tuesday, 25 August 2026, 11:59 PM** — exactly *two weeks* from issue (11 August 2026). Late submissions lose 10% per day unless a documented excuse is provided in advance (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions.

8.1 Conceptual

Q1. A system gates every interrupt request through two independent switches: a single global bit `IE` inside the processor status register, and a per-device enable bit (`KIRQ` for the keyboard, `DIRQ` for the display) inside that device's control register. Explain why *both* levels are needed rather than a single global switch. Give one concrete scenario where a programmer would want to (a) clear `PS.IE` globally while leaving every per-device enable set, and (b) clear a single device's own enable bit while leaving `PS.IE`= 1.

Q2. Compare two designs for resolving "which device raised the interrupt?": **polling the sources** and **vectored interrupts**. What exactly does the interrupt-vector table store, and why does the vectored design achieve *constant-time* dispatch regardless of device count? Under what conditions is polling the sources still an acceptable choice?

8.2 Numerical

Q3. A 1 GHz CPU reads input from a keyboard that produces 10 keystrokes per second (one every 0.1 s). The `READWAIT` polling loop is 2 instructions per failed iteration (`Test KIN` plus the failed `Branch`). Following the method of the lecture's worked example, compute how many `READWAIT` iterations the CPU executes *between* two consecutive keystrokes. State what useful work the CPU accomplishes on each of those iterations.

Q4. A disk transfers a 12 KB block to memory, one 4-byte word at a time, using interrupt-driven I/O. Each ISR invocation costs 25 clock cycles of CPU time.

(a) How many ISR invocations does the whole block transfer require?

(b) Re-design the same transfer using a DMA controller whose setup costs 40 clock cycles and whose single completion interrupt costs 25 clock cycles. How many CPU events does DMA require in total?

(c) The CPU runs at 2 GHz. Convert both totals into CPU time (in microseconds) and state, in one sentence, which technique's cost is scale-independent in block size and which is not.

8.3 Design

Q5. A system has three interrupt-capable devices: a fire alarm, a keyboard, and a printer. The fire alarm is safety-critical and must never be masked; the keyboard is used continuously; the printer is used occasionally. You are given the `IPENDING/IENABLE` register pair and a priority encoder from the lecture's design.

(a) Assign each device a priority (1 = highest) and give the `IENABLE` mask value that keeps the fire alarm always enabled while allowing you to silence the keyboard and printer during a critical section.

(b) The fire alarm and the keyboard both raise a request on the same cycle while the printer is idle and interrupt nesting is disabled. Describe, step by step using the lecture's five-step handling scenario, the complete sequence from request through `RTI`, including what happens to the keyboard's request while the alarm's ISR runs.

Q6. Design the CPU-side setup for a 64 KB disk-to-memory DMA transfer, again 4 bytes per word.

(a) State the value that must be written into the DMA controller's word-count register.

(b) Describe the sequence of events after the CPU starts the DMA: who becomes bus master and via what mechanism, how the block is moved, and exactly how many times the CPU is interrupted for the whole transfer.

(c) Compare the three DMA transfer modes — **burst**, **cycle stealing**, and **transparent** — in terms of how long each holds the bus, and state what the CPU loses (or does not lose) in each mode.